

ShopEasy Term Project – Reflection Report

Name: Kerem Tuna

Student ID: 210717038

Course: SE3004 Software Testing

1. Bugs Found by Each Technique

In this project, I didn't encounter any real defects in the starter code—after writing the tests, everything passed. Still, each testing technique would catch different types of mistakes if the code were faulty.

Specification-based testing (Task 1): Testing `PriceCalculator` against specifications is useful for catching formula errors. For example, if the discount and tax steps were swapped, tests like `testTypical108` (100, 10, 20 → 108) would fail. Invalid partition tests also revealed that the calculator accepted negative base prices before Task 3. After adding assertions, these invalid inputs were caught as `AssertionError` by the contract tests.

Structural testing with JaCoCo (Task 2): This helped identify which branches in `ShoppingCart` were not covered. Initially, I didn't test the case where a product exists in the cart but has the wrong ID in `updateQuantity`. JaCoCo highlighted this missing branch, so I added `updateQtyWrongId` and reached at least 80% branch coverage. Structural testing doesn't always find logic bugs but is effective for revealing missing tests. I also ran PIT mutation testing (see Task 2 bonus below).

Design by Contract (Task 3): Before asserts were added, invalid inputs such as a negative base price were not rejected at runtime. After Task 3, `ContractTest` and invalid-partition tests expect `AssertionError`. Assertions also document pre- and post-conditions directly in the production code.

Property-based testing with jqwik (Task 4): Properties like monotonicity (higher discount should never increase the total price) would catch errors like a sign mistake in the discount formula across hundreds of input combinations, not just a few hand-picked examples. In my run, jqwik didn't report any failures.

Mocks (Task 5): Using mocks for `OrderProcessor` let me test behavior without relying on a real warehouse or payment API. For instance, I verified that when `isAvailable` returns false, `charge` is never called. If the payment logic were triggered before the inventory

check, the `inventoryFail` test would fail. However, mocks do not test the correctness of the real `InventoryService` implementation.

2. Most Effective Technique per Unit of Effort

For me, specification-based testing combined with structural coverage was the most efficient. Task 1 helped me understand the price formula and write clear examples. Task 2 then showed gaps in `ShoppingCart` with JaCoCo, which I could fix with just one or two additional tests.

Property-based testing was powerful but took more time to learn (jqwik annotations, `Assume.that`, `@Provide`). Mockito was very helpful for testing `OrderProcessor`, but it only applied to that one class with external dependencies.

If I had limited time, I would prioritize: specification-based tests and branch coverage first, then contracts, followed by mocks, and finally property-based testing.

Task 2 bonus — PIT mutation testing: I ran `mvn test org.pitest:pitest-maven:mutationCoverage`. On the first run, `ShoppingCart` had 95% mutation coverage (20 of 21 killed). Two surviving boundary mutants were instructive. First, in `ShoppingCart.updateQuantity`, PIT changed `quantity <= 0` to `quantity < 0`. My test `updateQtyInvalid` still passed because `CartItem.setQuantity(0)` also throws, masking the faulty cart-level guard. I added `updateQtyZeroWrongProduct`, which passes quantity 0 with a product id not in the cart and expects the message `Quantity must be > 0`; that kills the mutant because a weakened guard throws `Product not found` instead. Second, in `CartItem.setQuantity`, the same boundary change survived because no test called `setQuantity(0)` directly. I added `cartItemRejectsZeroQuantity` for that. After these tests, `ShoppingCart` reached 100% mutation coverage in PIT.

3. Testing Pyramid

Most of my test suite is unit tests at the bottom of the testing pyramid: fast tests for `PriceCalculator`, `ShoppingCart`, and `OrderProcessor` (with mocks). There are no integration tests involving a real database or HTTP API, and no end-to-end UI tests.

What's missing are integration tests connecting real components (for example, a real `InventoryService` with `OrderProcessor`) and a small set of system tests for the full checkout flow. I also did not implement performance or load tests.

4. If I Had One More Week

I would focus on integration tests for a full checkout process: using a real cart, a simple in-memory inventory, and a fake payment gateway that records charges. This would ensure mocks didn't hide any wiring issues.

Finally, I would test edge cases in `OrderProcessor` that I skipped: empty `customerId`, null cart, and a cart with many items to verify the inventory loop works correctly.

5. Trade-offs of Mockito

Mocking allowed me to test the decision logic of `OrderProcessor` in isolation, for example: returning null when stock fails, calling charge only when all items pass, and returning null if payment fails.

However, I couldn't test that a real payment provider actually charged the correct amount or that the database stock was updated. Mocking is less suitable when interactions between real components are important or when mocks mimic the real implementation so closely that the test loses meaning.